

DDA5001 Machine Learning

Cross Validation & Regularization

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen



Recap: Overfitting and its Catalysts

Overfitting

Fitting the data more than is needed

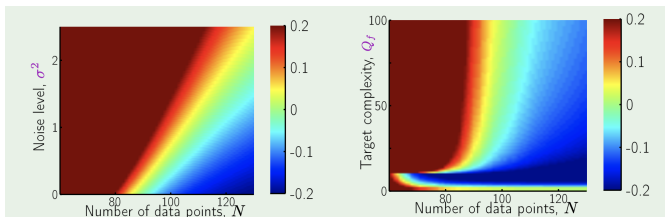


Figure: Color means overfitting level.

Overfitting: Catalysts

- ▶ **Number of training samples** increase, overfitting decreases.
- ▶ **Noise** in data increase, overfitting increases.
- ▶ **Model complexity** increases, overfitting increases.

Recap: Validation

Validation technique tries to estimate the out-of-sample error:

$$\underbrace{\text{Er}_{\text{out}}(f)}_{\text{validation estimates this quantity}} \leq \text{Er}_{\text{in}}(f) + \text{overfit penalty}.$$

Validation is used for **model selection** for avoiding overfitting.

The idea

Split the training set to another 'training set' and validation set.

Then, use the validation set for estimating Er_{out} .

Recap: Validation Error and Approximation of Er_{out}

Validation error:

$$\text{Er}_{\text{val}}(f') = \frac{1}{k} \sum_{i=1}^k e(f'(\mathbf{x}_i), y_i)$$

Estimate Er_{out} :

$$\text{Er}_{\text{out}}(f') \leq \text{Er}_{\text{val}}(f') + \mathcal{O}\left(\frac{1}{\sqrt{k}}\right).$$

Restoring: After we have used the validation set to estimate the out-of-sample error, **re-train on the whole data set** to get $\hat{f}$. Using a reasonable guess from VC analysis, we have

$$\text{Er}_{\text{out}}(\hat{f}) \leq \text{Er}_{\text{out}}(f') \leq \text{Er}_{\text{val}}(f') + \mathcal{O}\left(\frac{1}{\sqrt{k}}\right).$$

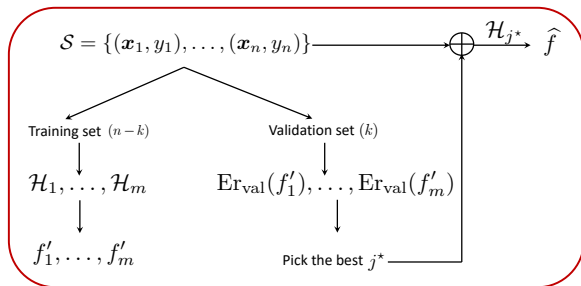
Recap: Validation for Model Selection

- **Setup:** Suppose we have m candidate hypothesis (can also be m different learning rate choices, etc)

$$\mathcal{H}_1, \dots, \mathcal{H}_m.$$

We can use the validation set to estimate the out-of-sample error by using $\text{Er}_{\text{val}}(f'_j)$ for each f'_j learned from those model spaces.

- **Selection:** Choose j^* such that $\text{Er}_{\text{val}}(f'_{j^*}) \leq \text{Er}_{\text{val}}(f'_j)$ for all j .
- **Restoring:** Train f on the whole set using model space $\mathcal{H}_{j^*}$, get $\hat{f}$.



Agenda

Validation — Continued

Regularization

Validation vs. Testing

- ▶ We call this “validation”, but how is it different from “testing”?
- ▶ Typically, validation is used to **make learning choices**, i.e., choosing hyper-parameters to avoid overfitting.

However,

The test data can never influence the training phase in any way.

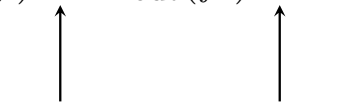
If it impacts the learning process, i.e., which final $\hat{f} \in \mathcal{H}$ we choose, then **it is no longer a test set,**

it becomes a validation set.

Validation Dilemma

Validation relies on the following chain of reasoning:

$$\text{Er}_{\text{out}}(\hat{f}) \approx \text{Er}_{\text{out}}(f') \approx \text{Er}_{\text{val}}(f')$$


small k large k

- All we need to do is set k so that it is **simultaneously** small and large... A dilemma we face for choosing k .

Is it possible? Yes.

▪

Cross Validation

Cross Validation: Leave One Out

- ▶ We need k to be small, so set $k = 1$.

$$\mathcal{S}_{\text{train}}^j = \{(\mathbf{x}_1, y_1), \dots, \cancel{(\mathbf{x}_j, y_j)}, \dots, (\mathbf{x}_n, y_n)\}$$

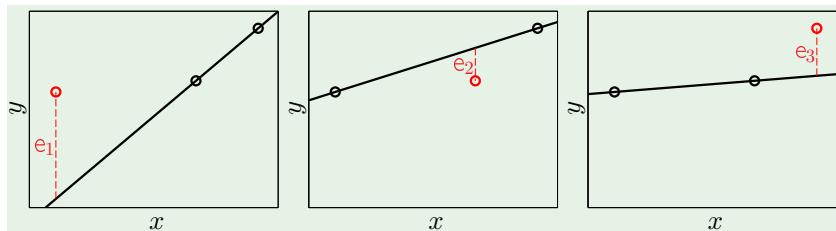
- ▶ Learn f'_j using $\mathcal{S}_{\text{train}}^j$. f'_j should have the almost the same quality as that of $\hat{f}$.
- ▶ Validation error: $\text{Er}_{\text{val}}(f'_j) = e(f'_j(\mathbf{x}_j), y_j) := e_j$.
- ▶ Since $k = 1$, $\text{Er}_{\text{val}}(f'_j)$ is a terrible estimate of $\text{Er}_{\text{out}}(f'_j)$.

The idea: Repeat this for all possible choices of j , and then average them, giving the cross validation error:

$$\text{Er}_{\text{cv}} = \frac{1}{n} \sum_{j=1}^n e_j.$$

This approach is called **leave-one-out cross validation**.

Cross Validation: Example



$$\text{Er}_{\text{cv}} = \frac{1}{3}(e_1 + e_2 + e_3)$$

- The hope is that the n validation errors **together** (i.e., Er_{cv}) is somehow equivalent to estimating Er_{out} using the whole data set of size n , while at the same time train f'_j on $n - 1$ data points.

Cross Validation for Model Selection

- ▶ **Setup:** Suppose that we have m candidate model spaces $\mathcal{H}_1, \dots, \mathcal{H}_m$ (can also be m different learning rate choices, etc).
- ▶ We use cross validation to estimate Er_{out} of $\mathcal{H}_i$ for $i = 1, \dots, m$ by computing Er_{cv} of using $\mathcal{H}_i$.
- ▶ Choose i^* that has the smallest Er_{cv} over all i . Obtain $\hat{f} = f'_{i^*}$, as there is no need to do restoring (only one data point difference).

What is the potential drawback of leave-one-out cross validation?

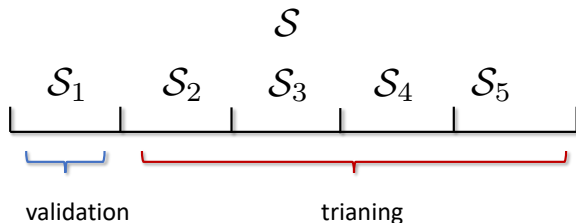
- ▶ For obtaining each Er_{cv} of using $\mathcal{H}_i$, we need to train n times on $n - 1$ samples each.
- ▶ In addition, for selecting the model $\mathcal{H}_{i^*}$, we need to repeat it for m times, requiring around **mn rounds of training on $n - 1$ data points**.
- ▶ When n is quite large, it can be computationally prohibitive.

Cross Validation: Leave More Out

k -fold cross validation: Choose a batch of data points for validation rather than one point.

In k -fold cross validation, k is the number of folds and $k' = \frac{n}{k}$ is the size of the validation set

Example: $k = 5$



- ▶ Iterate over all 5 choices of validation set and average. So, we only need to train k times on $n - \frac{n}{k}$ samples each.
- ▶ For cross validation with m hypothesis spaces, we need mk rounds of learning on $n - \frac{n}{k}$ samples each.

Common choice: $k = 5, 10$.

Leave More Out: Remarks

- ▶ For k -fold cross validation, the estimate depends on the particular choice of partition.
- ▶ When using k -fold cross validation for **classification**, one should ensure that each of the sets $\{\mathcal{S}_j\}$ contain training data from each class **in almost the same proportion** as in the full data set.
- ▶ It is common to form several estimates based on different **random partitions**.

Agenda

Validation — Continued

Regularization

Regularization

- Validation is to estimate Er_{out} , and then adjust hyper-parameters.

The regularization is another weapon for eliminating overfitting, which penalizes the model complexity using penalty $\Omega(\mathcal{H})$:

$$\text{Er}_{\text{out}}(f) \leq \text{Er}_{\text{in}}(f) + \Omega(\mathcal{H}), \quad \forall f \in \mathcal{H}$$

- From VC analysis, it is better to fit the data using the simplest workable $\mathcal{H}$. However, it is hard to determine such a perfect $\mathcal{H}$.
- One further heuristic extrapolation: How about use a rich/complex enough $\mathcal{H}$, but choose a 'simple' (the simplest workable) f from $\mathcal{H}$. Thus, we can choose a penalty $\Omega(f)$ to penalize the complexity of an individual f .
- Instead of minimizing $\text{Er}_{\text{in}}(f)$ alone, regularization amounts to minimizing simultaneously the training error and the complexity penalty, i.e.,

$$\min_{f \in \mathcal{H}} \text{Er}_{\text{in}}(f) + \Omega(f).$$

Least Square Revisited

Learning problem for least squares (linear regression)

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta} \in \mathbb{R}^d}{\operatorname{argmin}} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2,$$

where $\mathbf{X} \in \mathbb{R}^{n \times d}$.

When the data matrix $\mathbf{X}$ has full column rank (i.e., when its rank is d), we have a unique closed-form solution for LS:

$$\hat{\boldsymbol{\theta}} = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y}$$

- ▶ How about $n < d$?
- ▶ What is such a case? **Overfitting** occurs as d begins to exceed the number of samples n . We will get zero training error, but large test error.

Regularization Technique I: ℓ_2 -regularization

One candidate **regularizer**:

$$\Omega(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_2^2.$$

The ℓ_2 -regularized LS is:

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta} \in \mathbb{R}^d}{\operatorname{argmin}} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_2^2$$

- ▶ The most direct way to reduce complexity is to let $\boldsymbol{\theta}$ have many zeros (recall its ‘VC dimension’ is d). But this is too hard a constraint. ℓ_2 -regularizer is to make some parameters small (close to zero).
- ▶ $\lambda > 0$ is the **regularization parameter** (a hyper-parameter) that controls the trade-off between underfitting and overfitting.
 - Too large λ results in underfitting.
 - Too small λ may lead to overfitting.
- ▶ **Validation** technique can be used to choose this hyper-parameter.
- ▶ We can apply ℓ_2 -regularization to logistic regression too.

Solution of ℓ_2 -regularization

Let

$$\mathcal{L}(\boldsymbol{\theta}) = \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda\|\boldsymbol{\theta}\|_2^2$$

Expanding the ℓ_2 -norms yields

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}) &= (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) + \lambda\boldsymbol{\theta}^\top \boldsymbol{\theta} \\ &= \mathbf{y}^\top \mathbf{y} + \boldsymbol{\theta}^\top (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}) \boldsymbol{\theta} - 2\boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{y}\end{aligned}$$

Taking the gradient

$$\nabla \mathcal{L}(\boldsymbol{\theta}) = 2(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}) \boldsymbol{\theta} - 2\mathbf{X}^\top \mathbf{y}$$

Setting the gradient to zero gives

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

ℓ_2 -regularization vs. Vanilla Least Squares

Least squares:

$$\begin{aligned}\hat{\boldsymbol{\theta}} &= \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 \\ &= \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y} - \text{only for full column rank case}\end{aligned}$$

ℓ_2 -regularization:

$$\begin{aligned}\hat{\boldsymbol{\theta}} &= \operatorname{argmin}_{\boldsymbol{\theta} \in \mathbb{R}^d} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 + \lambda \|\boldsymbol{\theta}\|_2^2 \\ &= \left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}\right)^{-1} \mathbf{X}^\top \mathbf{y}\end{aligned}$$

The advantage of ℓ_2 -regularization:

$$\underbrace{\left(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}\right)^{-1}}_{\text{always invertible}}$$

Algebraically explained why ℓ_2 -regularization is useful.

Weight Decay

Weight decay is an important technique in machine learning. It is used almost everywhere in the training of neural networks.

- ▶ Weight decay is proposed as a technique for directly decaying the parameter (weight) θ during the algorithm process. It has the form:

$$\theta_{k+1} = (1 - \lambda)\theta_k - \mu \nabla \mathcal{L}(\theta_k),$$

where λ defines the rate of the weight decay per step.

- ▶ It is easy to see that if $1 - \lambda \in (0, 1)$, the weight parameter θ is decaying at each iteration, thus the name weight decay.

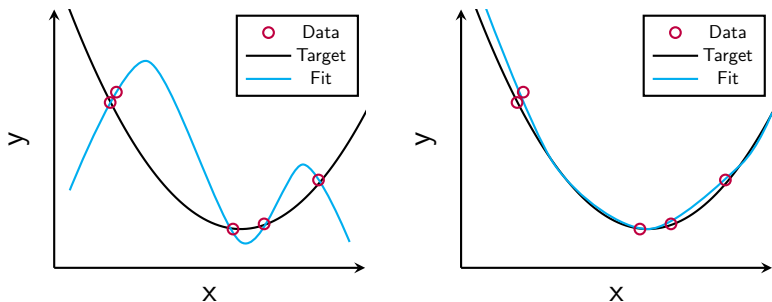
Indeed, we can verify that weight decay is equivalent to applying gradient descent to the ℓ_2 -regularized problem:

$$\min_{\theta \in \mathbb{R}^d} \mathcal{L}(\theta) + \frac{\lambda'}{2} \|\theta\|_2^2, \quad \text{with} \quad \lambda' = \frac{\lambda}{\mu}.$$

However, this is **NOT** the case in the Adam algorithm (later); see [1].

[1] Loshchilov, I., & Hutter, F. (2017). Decoupled weight decay regularization. ICLR 2019.

Regularization as a Cure for Overfitting



- ▶ Left: Using fourth-order polynomial **without regularization**.
- ▶ Right: Using fourth-order polynomial **with regularization (weight decay)**.

⇒ Next lecture: Another regularization technique using ℓ_1 -norm and concluding overfitting section.